

tf.init_scope Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/init_scope

A context manager that lifts ops out of control-flow scopes and function-building graphs.

Function Signatures

```
tf.init_scope() -> Iterator[None]
```

Code Examples

```
@tf_contextlib.contextmanager  
tf.init_scope() -> Iterator[None]
```

```
@tf_contextlib.contextmanager
```

```
tf.init_scope() -> Iterator[None]
```

```
tf.compat.v1.enable_eager_execution()  
@tf.function  
def func():  
    # A function constructs TensorFlow graphs,  
    # it does not execute eagerly.  
    assert not tf.executing_eagerly()  
    with tf.init_scope():  
        # Initialization runs with eager execution enabled  
        assert tf.executing_eagerly()
```

```
tf.compat.v1.enable_eager_execution()  
@tf.function  
def func():  
    # A function constructs TensorFlow graphs,  
    # it does not execute eagerly.  
    assert not tf.executing_eagerly()  
    with tf.init_scope():  
        # Initialization runs with eager execution enabled  
        assert tf.executing_eagerly()
```

Documentation

A context manager that lifts ops out of control-flow scopes and function-building

graphs.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.init_scope`

Used in the notebooks

- Better performance with `tf.function`
 - Preprocessing data with TensorFlow Transform
-

There is often a need to lift variable initialization ops out of control-flow scopes, function-building graphs, and gradient tapes. Entering an `init_scope` is a mechanism for satisfying these desiderata. In particular, entering an `init_scope` has three effects:

(1) All control dependencies are cleared the moment the scope is entered; this is equivalent to entering the context manager returned from `control_dependencies(None)`, which has the side-effect of exiting control-flow scopes like `tf.cond` and `tf.while_loop`.

(2) All operations that are created while the scope is active are lifted into the lowest context on the `context_stack` that is not building a

graph function. Here, a context is defined as either a graph or an eager context. Every context switch, i.e., every installation of a graph as the default graph and every switch into eager mode, is logged in a thread-local stack called `context_switches`; the log entry for a context switch is popped from the stack when the context is exited. Entering an `init_scope` is equivalent to crawling up `context_switches`, finding the first context that is not building a graph function, and entering it. A caveat is that if graph mode is enabled but the default graph stack is empty, then entering an `init_scope` will simply install a fresh graph as the default one.

(3) The gradient tape is paused while the scope is active.

When eager execution is enabled, code inside an `init_scope` block runs with eager execution enabled even when tracing a `tf.function`. For example:

Raises